

Aula 04 – Linguagens, Autômatos, Expressões Regulares e Scanners

Prof. Dr. Amaury Antônio de Castro Junior

Compiladores I

2020/2

Tópicos

- ▶ Conceitos básicos de linguagens formais (revisão)
- ▶ Expressões regulares
- ▶ Especificação léxica de linguagens de programação
- ▶ Usando autômatos finitos para reconhecer expressões regulares
- ▶ Scanners e tokens

Especificação de Linguagens de Programação

- ▶ Desde a década de 60, a sintaxe de linguagens de programação é especificada por uma gramática formal
 - ▶ ALGOL 60: primeira linguagem especificada com notação BNF (Backus-Naur Form ou Backus-Normal Form) em 1959
 - ▶ Ajuda da área de linguística (Chomsky)

Exemplo: Gramática de uma Mini Linguagem

- ▶ `program ::= statement | program statement`
- ▶ `statement ::= assignStmt | ifStmt`
- ▶ `assignStmt ::= id = expr ;`
- ▶ `ifStmt ::= if ( expr ) stmt`
- ▶ `expr ::= id | int | expr + expr`
- ▶ `id ::= a | b | c | i | j | k | n | x | y | z`
- ▶ `int ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9`

Produções

- ▶ As regras de uma gramática são chamadas de produções
- ▶ As regras de produção contêm
 - ▶ Símbolos não terminais: variáveis gramaticais (program, statement, id, etc.)
 - ▶ Símbolos terminais: sintaxe concreta que aparece nos programas (a, b, c, 0, 1, if, (,), ...)
- ▶ Significado de nonterminal ::= <sequência de terminais e não terminais>
- ▶ Em uma derivação, uma instância de não-terminal pode ser substituída pela sequência de terminais e não-terminais à direita da regra de produção
- ▶ Geralmente, existem duas ou mais produções para um não-terminal – você pode usar qualquer uma delas em diferentes partes da derivação

Notações Alternativas

- Existem diversas notações populares para especificação de regras de produção; todas com o mesmo significado

`ifStmt ::= if ( expr ) stmt`

`ifStmt → if ( expr ) stmt`

`<ifStmt> ::= if ( <expr> ) <stmt>`

Exemplo de Derivação - Exercício

`a = 1 ; if ( a + 1 ) b = 2 ;`

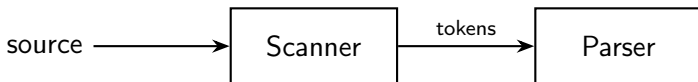
- ▶ `program ::= statement | program statement`
- ▶ `statement ::= assignStmt | ifStmt`
- ▶ `assignStmt ::= id = expr ;`
- ▶ `ifStmt ::= if ( expr ) stmt`
- ▶ `expr ::= id | int | expr + expr`
- ▶ `id ::= a | b | c | i | j | k | n | x | y | z`
- ▶ `int ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9`

Parsing

- ▶ Parsing: reconstruir a sequência de derivações (estrutura sintática) de um programa
- ▶ Em princípio, um único reconhecedor poderia trabalhar diretamente sobre a gramática concreta, caracter por caracter
- ▶ Na prática, isso não é feito

Parsing & Scanning

- ▶ Em compiladores reais o reconhecedor é dividido em duas fases:
 - ▶ Analisador Léxico (Scanner): traduz a cadeia de caracteres de entrada em átomos (tokens)
 - ▶ Além disso, identifica os erros léxicos, como caracteres e símbolos ilegais
 - ▶ Analisador Sintático (Parser): lê a cadeia de átomos e reproduz a sequência de derivações



Caracteres X Átomos (revisão)

► Cadeia de entrada

```
// this statement does very little  
if (x >= y) y = 42;
```

► Cadeia de átomos

```
IF LPAREN ID(x) GEQ ID(y)  
  RPAREN ID(y) BECOMES INT(42) SCOLON
```

Porque separar o Scanner e o Parser?

- ▶ Simplicidade e separação de compromissos (modularização)
 - ▶ O scanner esconde detalhes do parser (comentários, espaço em branco, arquivos de entrada etc.)
 - ▶ O parser é mais fácil de construir; possui fluxo de entrada mais simples (tokens)
- ▶ Eficiência
 - ▶ O scanner pode usar um projeto mais simples e rápido
 - ▶ (Mas ainda assim consome muito do tempo total de execução do compilador)

Átomos (Tokens)

- ▶ Ideia: precisamos definir um tipo de token distinto (classe léxica) para cada símbolo terminal distinto da linguagem
 - ▶ Devemos examinar a gramática para encontrar essas classes
- ▶ Alguns átomos podem ter atributos
 - ▶ Exemplos: o átomo `CONST_INT` terá o valor do número inteiro como um atributo; identificadores terão uma string com o atributo do átomo `ID`

Átomos mais Comuns em Linguagens de Programação

- ▶ Operadores & Pontuação
 - ▶ `+ - * / ( ) { } [ ] ; : :: < <= == = != ! ...`
 - ▶ Cada um desses átomos pertence a uma classe léxica distinta
- ▶ Palavras Reservadas (Keywords)
 - ▶ `if while for goto return switch void ...`
 - ▶ Cada um desses átomos pertence a uma classe léxica distinta (não identificadores e nem strings)
- ▶ Identificadores
 - ▶ Uma classe léxica única ID, parametrizada pela cadeia que forma o ID
- ▶ Constantes Inteiras
 - ▶ Uma classe léxica única INT, parametrizada pelo valor da constante
- ▶ Outras constantes, etc.

Princípio da Correspondência mais Longa

- ▶ Na maioria das linguagens, o scanner deve escolher a cadeia de caracteres mais longa possível para compor o próximo token, se houver uma opção.
- ▶ Exemplo:

```
return maybe != iffy;
```

- ▶ devem ser identificados 5 tokens
 - ▶ i.e., != é um token e não dois;
 - ▶ “iffy” é um ID, não um IF seguido de um ID(fy).

```
RETURN ID(maybe) NEQ ID(iffy) SCOLON
```

Teoria das Linguagens Formais e Autômatos (revisão em um slide)

- ▶ Alfabeto: um conjunto finito de símbolos
- ▶ Palavra (String): uma sequência finita, e possivelmente vazia, de símbolos de um alfabeto
- ▶ Linguagem: um conjunto, muitas vezes infinito, de palavras
- ▶ Especificações finitas de linguagens (possivelmente infinitas)
 - ▶ Autômato – um reconhecedor; uma máquina que aceita todas as palavras de uma linguagem (e rejeita todas as outras palavras)
 - ▶ Gramática – um gerador; um sistema para produzir todas as palavras da linguagem (e nenhuma outra palavra a mais)
- ▶ Uma dada linguagem pode ser especificada por muitas gramáticas e autômatos distintos
- ▶ Uma dada gramática ou autômato especifica apenas uma linguagem

Expressões Regulares e Autômatos Finitos

- ▶ A componente léxica da maioria das linguagens de programação pode ser especificada com expressões regulares
 - ▶ Às vezes, alguns “truques” são necessários
- ▶ Os tokens podem ser reconhecidos por um autômato finito determinístico
 - ▶ Pode ser orientado por tabela ou construído à mão com base na gramática da componente léxica

Expressões Regulares (REs)

- ▶ Definidas sobre algum alfabeto Σ
- ▶ Para as linguagens de programação, o alfabeto é, em geral, ASCII or Unicode
- ▶ Se re é uma expressão regular, $L(re)$ é a linguagem (conjunto de palavras) geradas por re

REs Fundamentais

re	$L(\text{re})$	Descrição
a	$\{a\}$	Conjunto unitário, para cada símbolo a em Σ
ε	$\{\varepsilon\}$	Palavra vazia
$\emptyset$	$\{ \}$	Linguagem vazia

Operações sobre REs

- ▶ Precedência: $*$ (mais alto), concatenação, $|$ (mais baixo)
- ▶ Parênteses podem ser usados para agrupar REs quando necessário

re	$L(re)$	Descrição
rs	$L(r)L(s)$	Concatenação
$r s$	$L(r) \cup L(s)$	Combinação (união)
r^*	$L(r)^*$	0 ou mais ocorrências (Fecho de Kleene)

Abreviações

- ▶ As operações básicas geram todas as expressões regulares possíveis, mas existem abreviações comuns usadas por conveniência. Principais exemplos:

Abrev.	Significado	Descrição
r^+	(rr^*)	1 ou mais ocorrências
$r?$	$(r \mid \varepsilon)$	0 ou 1 ocorrência
$[a-z]$	$(a b \dots z)$	1 caracter em um intervalo
$[abxyz]$	$(a b x y z)$	1 dos caracteres dados

Exemplos

re	Significado
+	Caracter simples +
!	Caracter simples !
=	Caracter simples =
!=	2 caracteres em sequência
<=	2 caracteres em sequência
xyzzzy	5 caracteres em sequência

Mais Exemplos (exercício)

re	Significado
[abc] ⁺	
[abc] [*]	
[0-9] ⁺	
[1-9][0-9] [*]	
[a-zA-Z][a-zA-Z0-9_ [*]	

Abreviações

- ▶ Muitos sistemas permitem abreviações para facilitar a escrita e a leitura de definições ou especificações:

`name ::= re`

- ▶ Restrição: abreviações não devem ser circulares (recursivas) nem direta e nem indiretamente (caso contrário, podem não ser regulares)

Exemplo

- ▶ Possível sintaxe para constantes numéricas
- ▶ `digit ::= [0-9]`
- ▶ `digits ::= digit+`
- ▶ `number ::= digits( . digits )? ([eE] (+|-)? digits)?`

Reconhecimento de REs

- ▶ Autômatos finitos podem ser usados para reconhecer palavras geradas por expressões regulares
- ▶ Podem ser construídas a mão ou automaticamente
- ▶ Não é totalmente direto, mas pode ser feito sistematicamente
- ▶ Ferramentas como Lex, Flex, Jlex, SLY entre outras fazem isso automaticamente, considerando um conjunto de REs

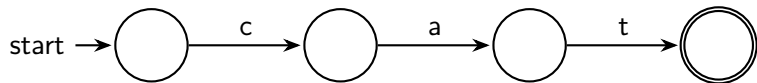
Autômato de Estados Finito (FSA) - 1/2

- ▶ Um conjunto finito de estados
- ▶ Um estado inicial
- ▶ Um ou mais estados finais
- ▶ Estados rotulados ou numerados
- ▶ Um conjunto de transições de um estado para outro
 - ▶ Cada uma rotulada com um símbolo de Σ , ou ε
- ▶ Operação baseada na leitura de símbolos de entrada (em geral, caracteres)
 - ▶ Transições são escolhidas se estiverem rotuladas com o símbolo corrente
 - ▶ Transições vazias (ε -transition) podem ser executadas a qualquer momento
 - ▶ Palavra é aceita quando alcança o estado final e não tem mais entrada

Autômato de Estados Finito (FSA) - 2/2

- ▶ Scanner usa um FSA como uma subrotina – aceita a correspondência mais longa sempre que é chamada, mesmo que ainda haja caracteres na entrada; i.e., executa o FSA a partir da posição corrente na cadeia de entrada sempre que o scanner é chamado
- ▶ Palavra é rejeitada se nenhuma transição for possível, ou não houver mais caracteres na entrada e não tiver alcançado um estado final (DFA)

Exemplo: FSA for “cat”



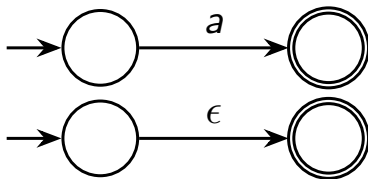
DFA vs NFA

- ▶ Deterministic Finite Automata (DFA)
 - ▶ Não há escolha de transição em nenhuma hipótese
- ▶ Non-deterministic Finite Automata (NFA)
 - ▶ Há escolha de transição em pelo menos um caso
 - ▶ Aceita se, de algum modo, alcança um estado final para uma determinada cadeia de entrada
 - ▶ Rejeita se não é possível alcançar o estado final

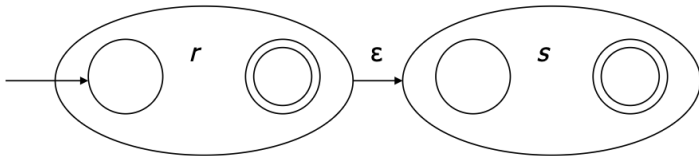
Autômatos Finitos em Scanners

- ▶ Prefiro um DFA por causa da velocidade (sem backtracking)
- ▶ Conversão de expressão regular para NFA é fácil
- ▶ Existe um procedimento bem definido para conversão de um NFA para um DFA equivalente

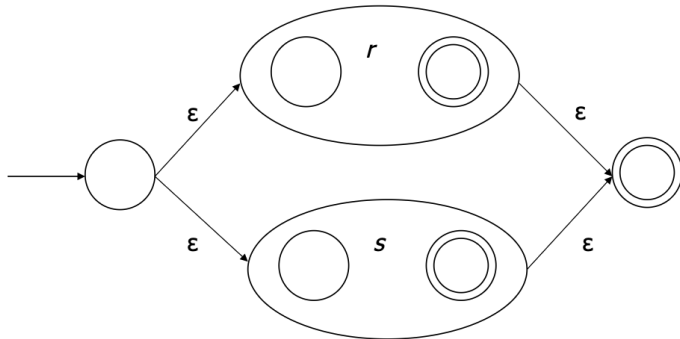
De RE to NFA: casos elementares



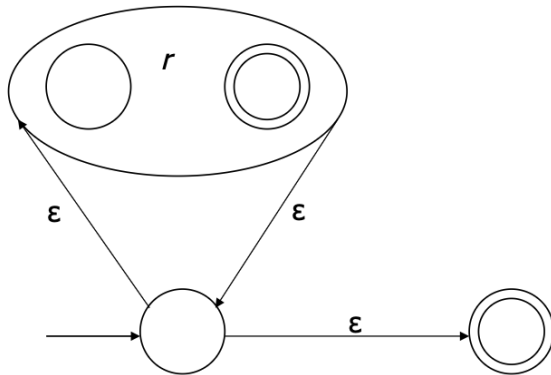
De RE to NFA: caso rs (concatenação)



De RE to NFA: caso $r | s$ (alternativa)



De RE to NFA: caso r^* (fecho)



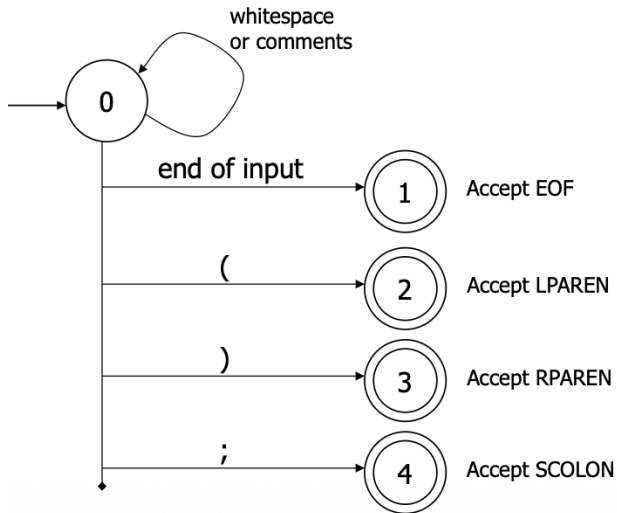
De NFA para DFA

- ▶ Construção do subconjunto
 - ▶ Construa um DFA a partir do NFA, em que cada estado do DFA representa um conjunto de estados do NFA
- ▶ Ideia-chave
 - ▶ O estado do DFA depois de ler alguma entrada é o conjunto de todos os estados que a NFA poderia ter alcançado depois de ler a mesma entrada
- ▶ Algoritmo: exemplo de uma computação de ponto fixo
 - ▶ Se o NFA tiver n estados, o DFA terá no máximo 2^n estados
 - ▶ $\Rightarrow$ O DFA é finito, pode ser construído em um número finito de etapas
 - ▶ O DFA resultante pode ter mais estados do que o necessário
- ▶ Consulte os livros para obter detalhes sobre construção e minimização

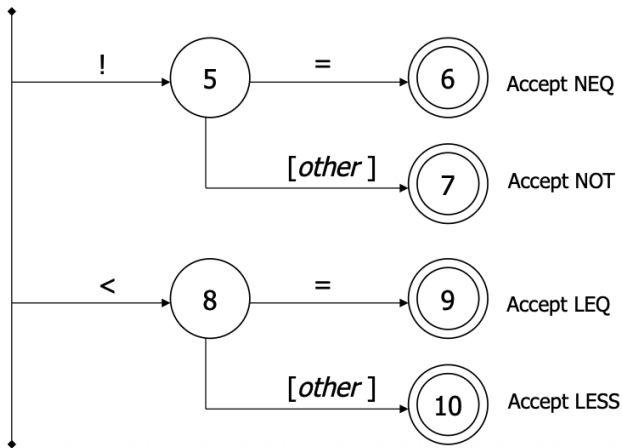
Exemplo: DFA para scanner à mão

- ▶ Ideia: mostrar um DFA escrito à mão para construções mais comuns de uma linguagem de programação
- ▶ Em seguida, use para construir um scanner à mão
- ▶ Configuração: o scanner é chamado sempre que o parser pedir um novo token
 - ▶ O scanner armazena a posição atual na entrada
 - ▶ A partir daí, use um DFA para reconhecer a maior sequência de entrada possível que compõe um token e retorne esse token

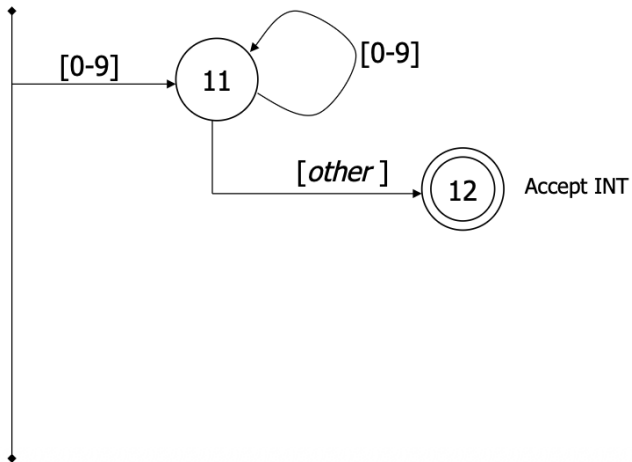
Exemplo de Scanner DFA (1)



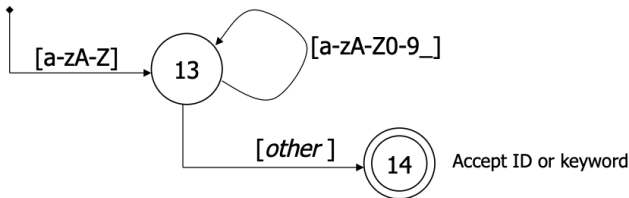
Exemplo de Scanner DFA (2)



Exemplo de Scanner DFA (3)



Estratégias para lidar com identificadores x palavras-chave



- ▶ Scanner à mão: verifique se os identificadores encontrados na entrada estão na tabela de palavras reservadas (boa aplicação de hashing)
- ▶ Scanner codificado: gerar o DFA fará as transições apropriadas para reconhecer palavras reservadas
 - ▶ Muitos estados, só que mais eficiente (nenhuma etapa extra de pesquisa é necessária)

Implementação de um Scanner à mão – Representação de Token

- Um token é uma estrutura simples e identificada

```
public class Token {  
    public int kind;      // token's lexical class  
    public int intVal;    // integer value if class = INT  
    public String id;     // actual identifier if class = ID  
  
    // lexical classes  
    public static final int EOF = 0;      // "end of file" token  
    public static final int ID = 1;       // identifier, not keyword  
    public static final int INT = 2;      // integer  
    public static final int LPAREN = 4;  
    public static final int SCOLN = 5;  
    public static final int WHILE = 6;  
    // etc. etc. etc. ...  
}
```

Exemplo de um Scanner Simples

```
// global state and methods
static char nextch; // next unprocessed input character

// advance to next input char
void getch() { ... }

// skip whitespace and comments
void skipWhitespace() { ... }
```

Método getToken() do Scanner (1)

```
// return next input token
public Token getToken() {
    Token result;
    skipWhiteSpace();
    if (no more input) {
        result = new Token(Token.EOF); return result;
    }
    switch(nextch) {
        case '(': result = new Token(Token.LPAREN); getch(); return result;
        case ')': result = new Token(Token.RPAREN); getch(); return result;
        case ';': result = new Token(Token.SCOLON); getch(); return result;
        // etc. ...
    }
}
```

getToken() (2)

```
case '!=': // ! or !=
    getch();
    if (nextch == '=') {
        result = new Token(Token.NEQ); getch(); return result;
    } else {
        result = new Token(Token.NOT); return result;
    }
case '<': // < or <=
    getch();
    if (nextch == '=') {
        result = new Token(Token.LEQ); getch(); return result;
    } else {
        result = new Token(Token.LESS); return result;
    }
// etc. ...
```

getToken() (3)

```
case '0': case '1': case '2': case '3': case '4':  
case '5': case '6': case '7': case '8': case '9':  
    // integer constant  
    String num = nextch;  
    getch();  
    while (nextch is a digit) {  
        num = num + nextch; getch();  
    }  
    result = new Token(Token.INT, Integer(num).intValue());  
    return result;  
...
```

getToken (4)

```
case 'a': ... case 'z':
case 'A': ... case 'Z': // id or keyword
    string s = nextch; getch();
    while (nextch is a letter, digit, or underscore) {
        s = s + nextch; getch();
    }
    if (s is a keyword) {
        result = new Token(keywordTable.getKind(s));
    } else {
        result = new Token(Token.ID, s);
    }
    return result;
```

Sobre o projeto

- ▶ Para o projeto, use um gerador de analisador léxico
- ▶ Sugestão: SLY Library (python).
- ▶ Alternativas:
 - ▶ Jflex
 - ▶ Bison
 - ▶ Lex
 - ▶ Yacc

Cenas dos próximos capítulos. . .

- ▶ Atividades do período:
 - ▶ Exercícios sobre expressões regulares e revisão de LFA, etc.
 - ▶ Primeira parte do projeto do compilador – o scanner
- ▶ Próximo tópico: Análise Sintática
 - ▶ Bom momento para ler os capítulos 3 e 4 do Livro do Appel.